LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack & Queue

Oleh:

Muhammad Hamdani **2510817310019**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MARET 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2 Stack & Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Hamdani
NIM : 2510817310019

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Fadhil Syahdama Mahatma Putra	Andreyan Rizky Baskara, S.Kom., M.Kom
NIM. 2410817210026	NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 1
A. Pembahasan 1
SOAL 2 2
A. Source Code 3
B.Output Program 7
C. Pembahasan 11
SOAL 3 13
A. Source Code 13
B.Output Program 17
C.Pembahasan 24
TAUTAN GIT HUB 26

DAFTAR GAMBAR

GAMBAR 1. Tampilan awal saat program di jalankan 7
GAMBAR 2. Memasukan nilai ke dalam tumpukan 8
GAMBAR 3. Menampilkan tumpukkan yang sudah dimasukkan 9
GAMBAR 4. Tampilan setelah melakukan delete 10
GAMBAR 5. Tampilan setelah memilih quit 11
GAMBAR 6. Tampilan awal program saal dijalankan 17
GAMBAR 7. Memasukkan Huruf ke Dalam queue 18
GAMBAR 8. Menampilkan Queue Yang Sudah Dimasukkan 19
GAMBAR 9. Melakukan Delete Pada Huruf yang Sudah Dimasukkan 20
GAMBAR 10. Tampilan Queue Setelah Melakukan Delete 21
GAMBAR 11. Tampilan mereset queue 22
GAMBAR 12. Tampilan Setelah melakukan reset dan Tampilan memilih quit 23

DAFTAR TABEL

Tabel 1. Source Code Soal Nomor 2 3
Tabel 2. Source Code Soal Nomor 3 13

SOAL 1

Apa perbedaan Stack dengan Queue?

A. Pembahasan

Stack dan Queue adalah struktur data yang mempunyai perbedaan dalam cara memasukkan/input (*push/enqueue*) dan mengeluarkan/output (*pop/dequeue*) data atau elemen yang ada.

Pada struktur data Stack, prinsip yang digunakan adalah **LIFO** (*Last in First Out*), yang mana data atau elemen yang paling terakhir dimasukkan (*push*) akan menjadi data atau elemen yang paling awal dikeluarkan (*pop*). Sebagai contoh, kita ambil analogi seperti ketika kita menumpuk piring kotor yang ada, piring kotor yang berada di tumpukkan paling bawah akan selesai dibersihkan paling akhir sedangkan, piring kotor yang terakhir ditumpuk akan menjadi yang paling awal untuk dibersihkan.

Sedangkan, pada struktur data Queue, prinsip yang digunakan adalah **FIFO** (*First in First Out*), yang mana data atau elemen yang pertama dimasukkan (enqueue) akan menjadi data atau elemen yang pertama untuk dikeluarkan (*dequeue*). Sebagai contoh kita ambil analogi seperti ketika kita mengantri untuk memesan gacoan, orang yang datang atau mendaftar pertama kali ke meja kasih ialah yang akan dilayani lebih dulu.

SOAL 2

Cobalah contoh program berikut, running dan analisis hasilnya.

```
int penuh()
{
    if(Tumpuk.atas == max-1)
        return 1;
    else
        return 0;
}

void input (int data)
{
    if (kosong()==1)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else if(penuh()==0)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data "<< Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else
        cout << "Tumpukan Penuh";
}
void hapus()
{
    if(kosong()== 0)
    {
        cout << "Data Teratas Sudah Terambil";
        Tumpuk.atas--;
    }
    else
    cout <<" Data Kosong";
}
void tampil()
{
    if (kosong()== 0)
    {
        for(int i = Tumpuk.atas; i>=0; i--)
        {
            cout <<"\nTumpukan Ke " << i << " = "
            <<Tumpuk.data[i];
        }
    }
    else
    cout << "Tumpukan Kosong";
}
void bersih ()
{
    Tumpuk.atas = -1;
    cout << "Tumpukan Kosong !";
}
```

A. Source Code

Tabel 1. Source Code Soal Nomor 2

```
#include <iostream>
#include <stdlib.h>

#define MAX 20
using namespace std;

struct Stack {
    int Atas;
    int Data[MAX];
};

Stack Tumpuk;

void Inisialisasi() {
    Tumpuk.Atas = -1;
}

int Kosong() {
    return (Tumpuk.Atas == -1);
}

int Penuh() {
    return (Tumpuk.Atas == MAX - 1);
}

void Push(int nilai) {
```

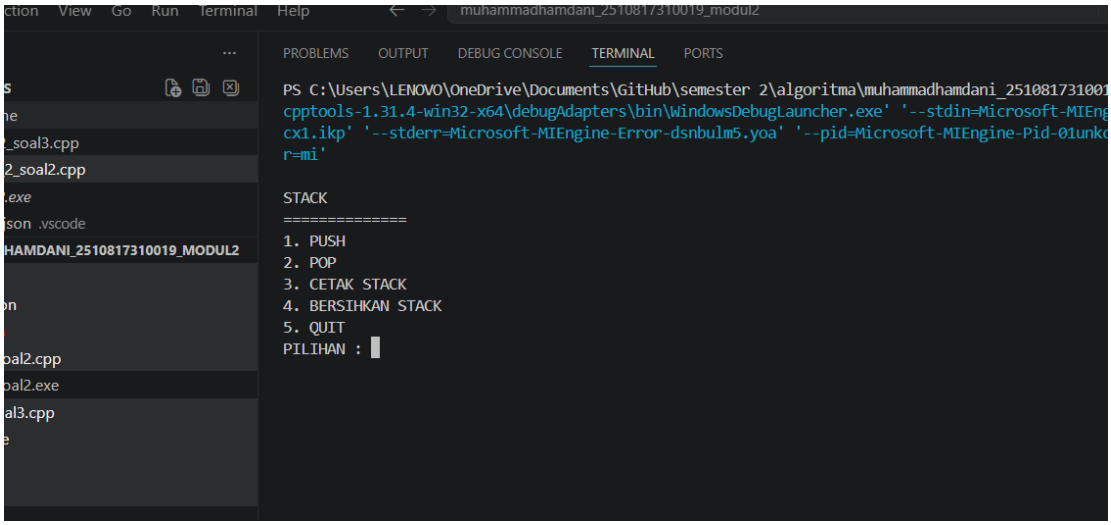
```
    if (!Penuh()) {
        Tumpuk.Atas++;
        Tumpuk.Data[Tumpuk.Atas] = nilai;
        cout << "Data " << nilai << " dimasukkan ke
dalam Stack." << endl;
    } else {
        cout << "Stack telah Penuh!!!" << endl;
    }
}

void Pop() {
    if (!Kosong()) {
        cout << "Data " << Tumpuk.Data[Tumpuk.Atas]
<< " diambil dari Stack." << endl;
        Tumpuk.Atas--;
    } else {
        cout << "Stack Kosong!!!" << endl;
    }
}

void Cetak_Stack() {
    if (!Kosong()) {
        // Mencetak dari atas ke bawah (LIFO)
        for (int i = Tumpuk.Atas; i >= 0; i--) {
            cout << "Tumpukan Ke " << i << " = " <<
Tumpuk.Data[i] << endl;
        }
    } else {
        cout << "Stack Kosong!!!" << endl;
```

```
56      }
57  }
58
59  int main() {
60      Inisialisasi();
61      int Pilihan, Data;
62
63      do {
64          //   Gunakan   banyak   baris   baru   jika
65  system("cls") bermasalah
66          // Ini cara paling aman agar tidak menyamping
67          cout << "\n\n\n\n\n";
686         cout << "STACK" << endl;
69          cout << "==============" << endl;
70          cout << "1. PUSH" << endl;
71          cout << "2. POP" << endl;
72          cout << "3. CETAK STACK" << endl;
73          cout << "4. BERSIHKAN STACK" << endl;
74          cout << "5. QUIT" << endl;
75          cout << "PILIHAN : ";
76
77          if (!(cin >> Pilihan)) {
78              cin.clear();
79              cin.ignore(1000, '\n');
80              continue;
81          }
82
83          switch (Pilihan) {
84              case 1:
```

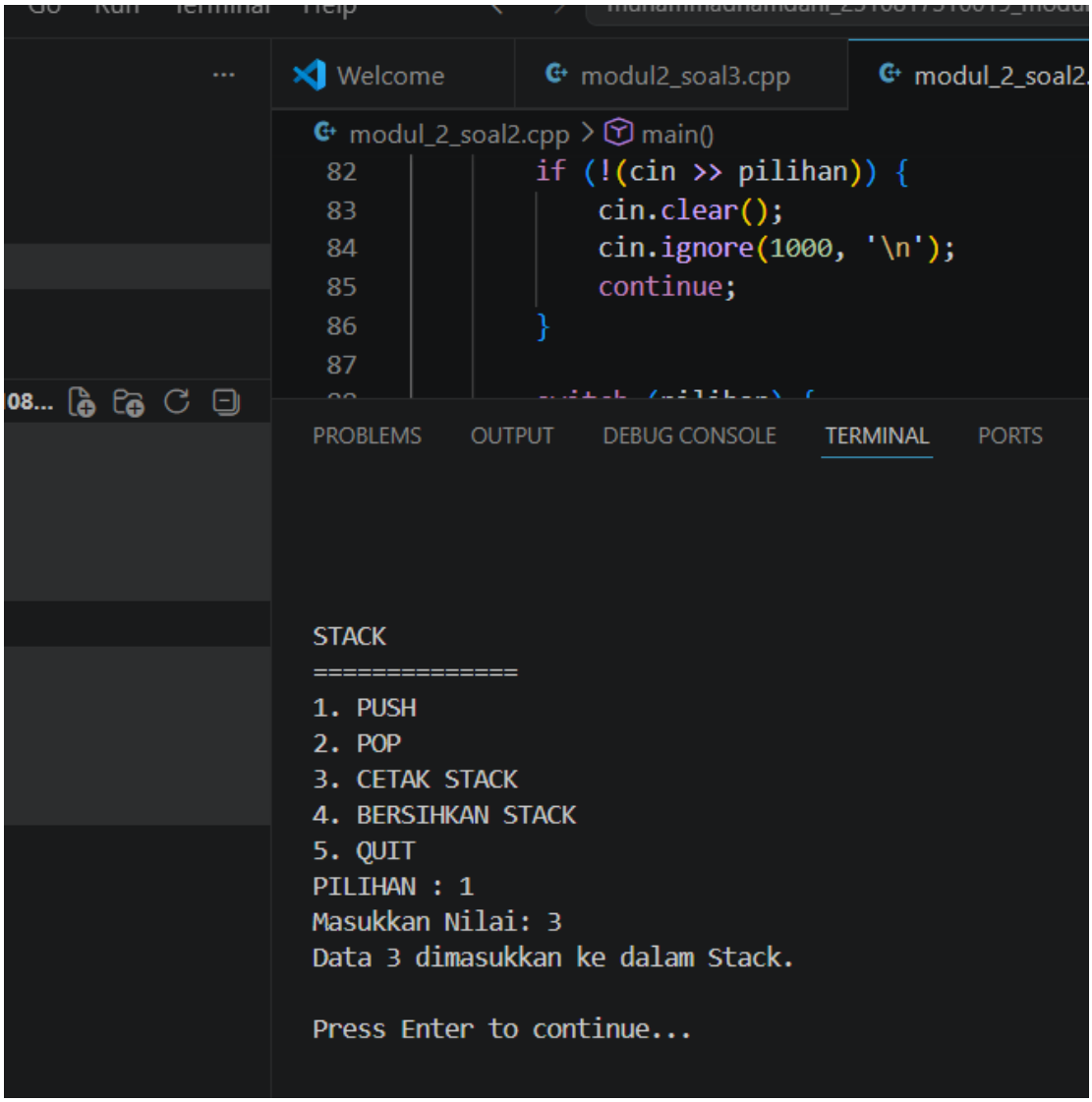
```
                cout << "Masukkan Nilai: "; cin >>
Data;
                Push(Data);
                break;
            case 2:
                Pop();
                break;
            case 3:
                Cetak_Stack();
                break;
            case 4:
                Inisialisasi();
                cout << "Stack telah dibersihkan!" <<
endl;
                break;
            case 5:
                cout << "TERIMA KASIH" << endl;
                break;
        }

        if (Pilihan != 5) {
            cout << "\nPress Enter to continue...";
            cin.ignore(1000, '\n'); // Membersihkan
sisa Enter sebelumnya
            cin.get();             // Menunggu tombol
Enter
        }

    } while (Pilihan != 5);
```

```
    return 0;
}
```

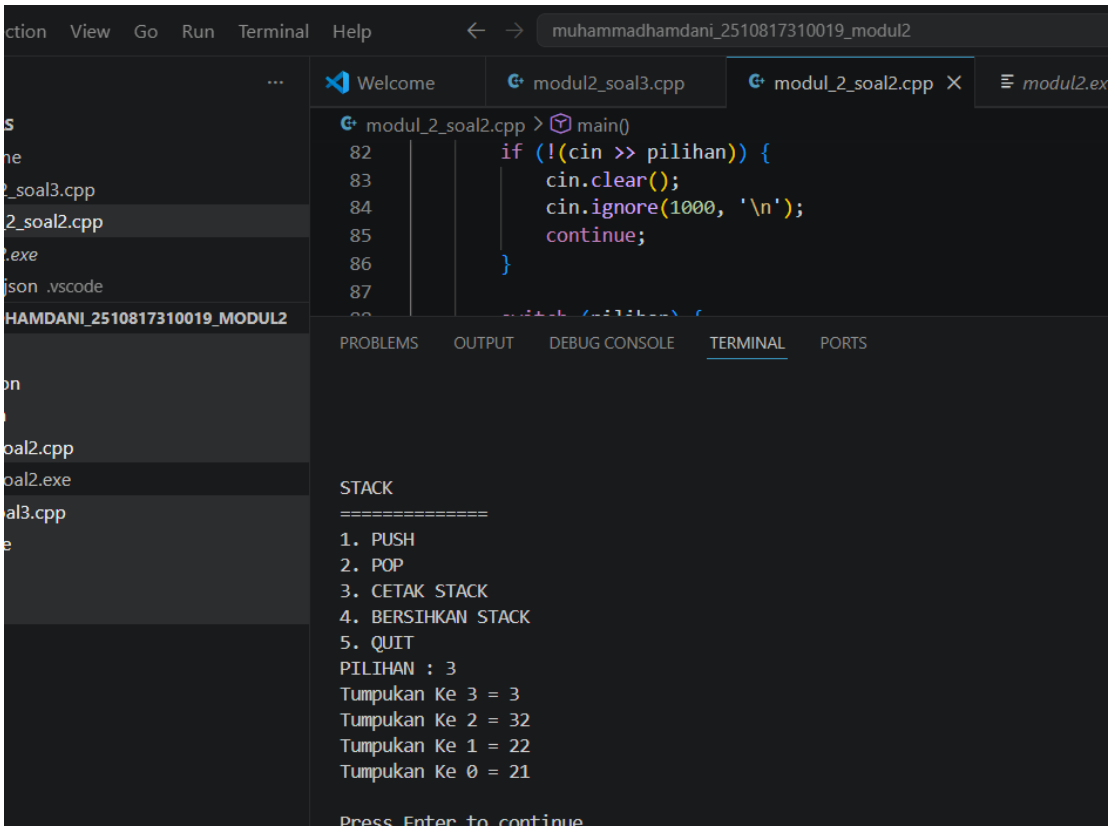
B.Output Program



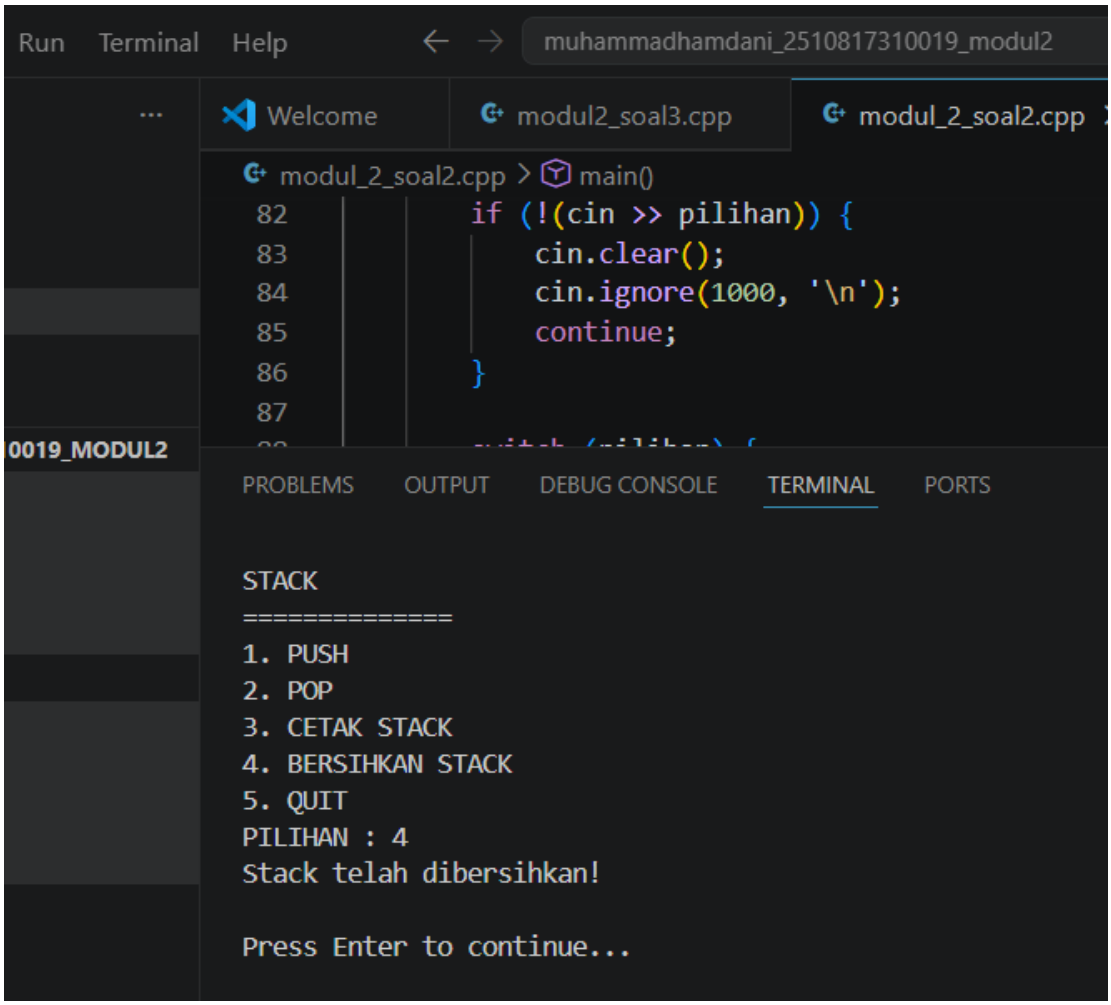
GAMBAR 1. Tampilan awal saat program di jalankan



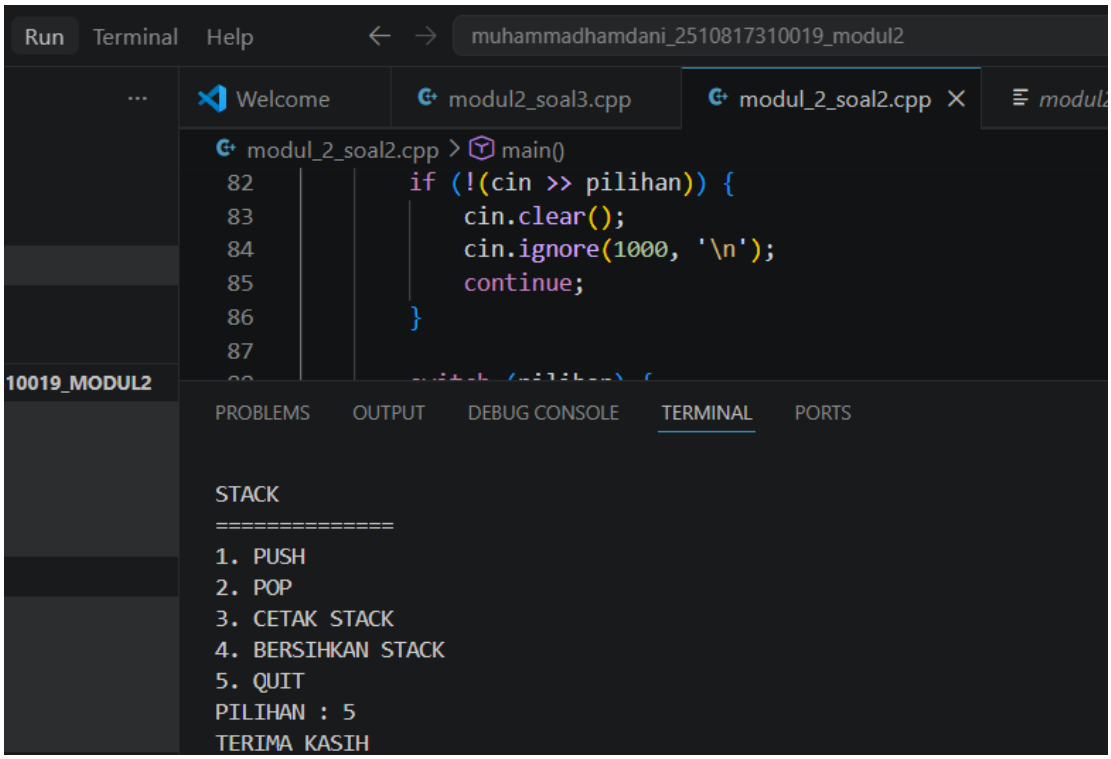
GAMBAR 2. Memasukan nilai ke dalam tumpukan



GAMBAR 3. Menampilkan tumpukkan yang sudah dimasukkan



GAMBAR 4. Tampilan setelah melakukan delete



GAMBAR 5. Tampilan setelah memilih quit

C. Pembahasan

Baris [1] sampai [2] terdapat #include <iostream> yang digunakan untuk proses input/output data, dan <stdlib.h> yang digunakan untuk perintah manajemen sistem seperti kontrol alur program.

Baris [4] terdapat perintah #define max 20. Ini digunakan untuk membuat konstanta bernama max dengan nilai 20, yang menentukan daya tampung maksimal elemen pada tumpukan.

Baris [7] sampai [10] mendefinisikan struct Stack. Ini adalah wadah untuk menyimpan data stack, di mana variabel int atas bertindak sebagai penunjuk posisi elemen paling atas (pointer), dan int data[max] adalah array untuk menyimpan nilai-nilai yang dimasukkan.

Baris [12] membuat variabel bernama Tumpuk yang bertipe Stack. Variabel inilah yang akan kita manipulasi sepanjang program.

Baris [14] sampai [17] terdapat fungsi void bersih(). Fungsinya untuk mengosongkan tumpukan dengan cara mengatur nilai Tumpuk.atas = -1. Secara logika, jika indeks berada di -1, maka tidak ada data yang terdaftar di dalam array.

Baris [19] sampai [27] terdapat fungsi pengecekan kosong() dan penuh(). Fungsi kosong() mengembalikan nilai 1 jika tumpukan benar-benar kosong (atas == -1), sedangkan fungsi penuh() mengembalikan nilai 1 jika indeks atas sudah mencapai batas maksimal array (max - 1).

Baris [29] sampai [41] terdapat fungsi void input(int data_baru). Fungsi ini merupakan implementasi dari operasi Push. Sesuai logika di gambar, program akan mengecek apakah tumpukan kosong atau belum penuh. Jika syarat terpenuhi, nilai atas akan bertambah (increment) dan data disimpan ke dalam array. Jika sudah mencapai batas, akan muncul pesan "Tumpukan Penuh".

Baris [43] sampai [50] terdapat fungsi void hapus(). Fungsi ini merupakan implementasi dari operasi Pop. Jika tumpukan tidak kosong, maka program akan menampilkan pesan bahwa data teratas diambil dan indeks atas akan dikurangi (decrement). Ini mengikuti prinsip LIFO (Last In First Out).

Baris [52] sampai [60] terdapat fungsi void tampil(). Fungsi ini menampilkan seluruh isi tumpukan. Digunakan perulangan for yang berjalan mundur (dari atas ke 0) agar pengguna dapat melihat data dari posisi paling atas hingga paling bawah.

Baris [62] sampai [105] terdapat fungsi main() sebagai pusat kendali. Program menggunakan do-while untuk terus menampilkan menu selama pengguna tidak memilih angka 5 (Quit). Terdapat fitur validasi if (!(cin >> pilihan)) untuk mencegah error jika pengguna salah memasukkan huruf, serta cin.ignore() dan cin.get() untuk memastikan transisi antar menu berjalan lancar dan rapi di terminal.

SOAL 3

A. Source Code

Tabel 2. Source Code Soal Nomor 3

```
#include <iostream>
#include <stdlib.h>

#define n 10
using namespace std;

// Deklarasi Fungsi
void INSERT();
void DELETE();
void CETAKLAYAR();
void Inisialisasi();
void RESET();

// Variabel Global
int PIL, F, R;
char PILIHAN[2], HURUF;
char Q[n];

int main() {
    Inisialisasi();
    do {
        cout << "\n\n\n\n\n";
        cout << "QUEUE" << endl;
        cout << "===============" << endl;
```

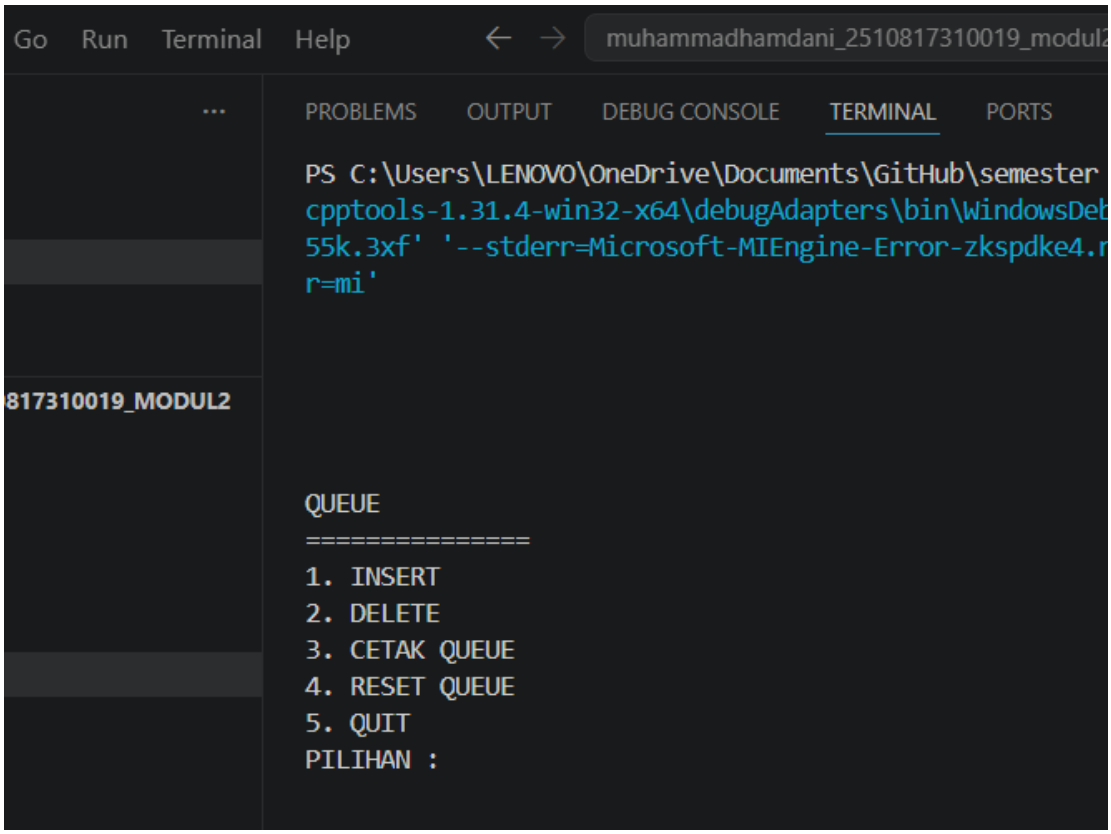
```
25          cout << "1. INSERT" << endl;
26          cout << "2. DELETE" << endl;
27          cout << "3. CETAK QUEUE" << endl;
28          cout  <<  "4.  RESET  QUEUE"  <<  endl;  //
29  Menambahkan menu reset
30          cout << "5. QUIT" << endl;
31          cout << "PILIHAN : "; cin >> PILIHAN;
32
33          PIL = atoi(PILIHAN);
34
35          switch (PIL) {
36              case 1:
37                  INSERT();
38                  break;
39              case 2:
40                  DELETE();
41                  break;
42              case 3:
43                  CETAKLAYAR();
44                  break;
45              case 4:
46                  RESET();
47                  cout    <<    "Antrean    Berhasil
48  Dikosongkan (Reset)." << endl;
49                  break;
50              default:
51                  cout << "TERIMA KASIH" << endl;
52                  break;
53          }
```

```
54
55          if (PIL < 5) {
56              cout << "\npress any key to continue" <<
57  endl;
58              cin.ignore(1000, '\n');
59              cin.get();
60          }
61
62      } while (PIL < 5);
63
64      return 0;
65  }
66
67  void Inisialisasi() {
686     F = -1;
69      R = -1;
70  }
71
72  void RESET() {
73      F = -1;
74      R = -1;
75  }
76
77  void INSERT() {
78      if (R == n - 1) {
79          cout << "Antrean Penuh (Overflow)!" << endl;
80      } else {
81          cout << "Masukkan Huruf: "; cin >> HURUF;
82          if (F == -1) F = 0;
```

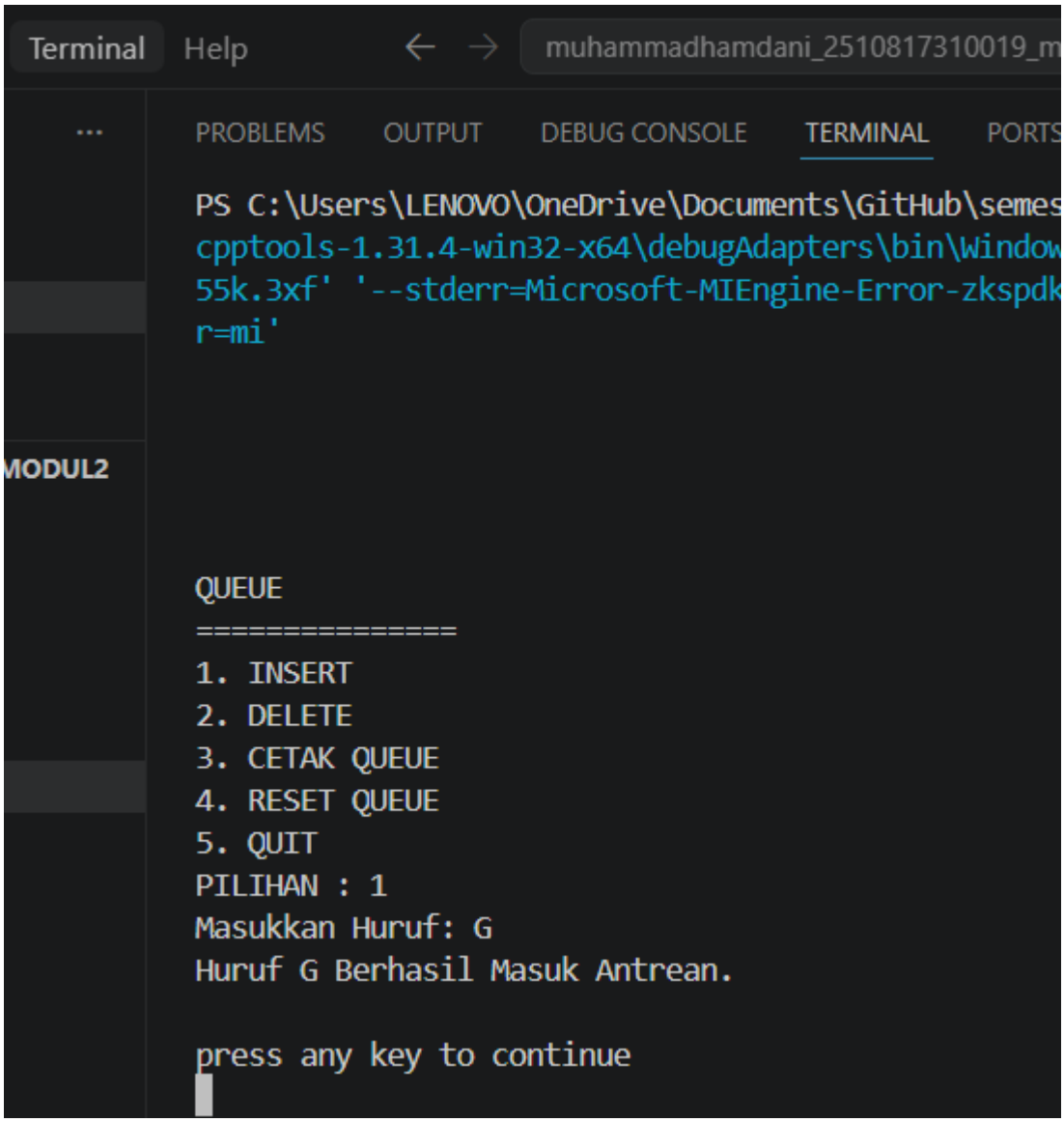
```
        R++;
        Q[R] = HURUF;
        cout << "Huruf " << HURUF << " Berhasil Masuk
Antrean." << endl;
    }
}

void DELETE() {
    if (F == -1 || F > R) {
        cout << "Antrean Kosong (Underflow)!" <<
endl;
        RESET(); // Otomatis reset jika sudah benar-
benar kosong
    } else {
        cout << "Huruf " << Q[F] << " Dikeluarkan
dari Antrean." << endl;
        F++;
    }
}

void CETAKLAYAR() {
    if (F == -1 || F > R) {
        cout << "Antrean Kosong!" << endl;
    } else {
        // Menampilkan isi antrean dari Front sampai
Rear
        for (int i = F; i <= R; i++) {
            cout << "Queue ke- " << i << " = " <<
Q[i] << endl;
```

```
            }
        }
}
```

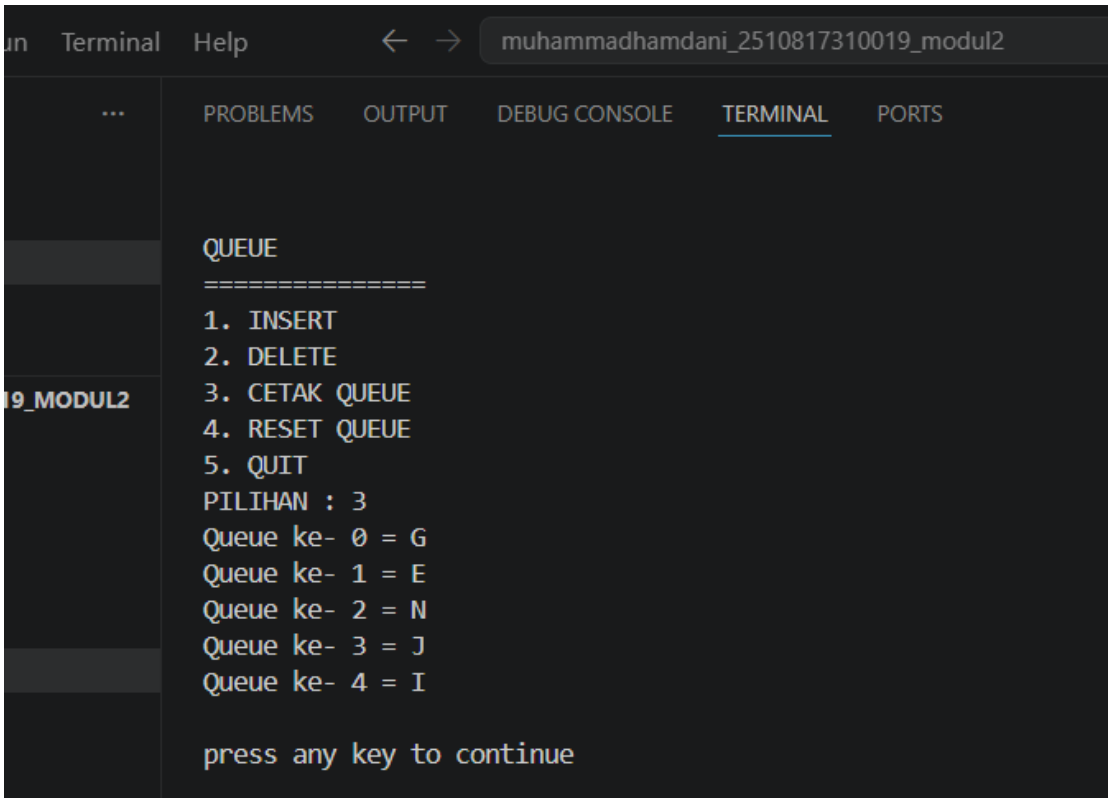
B.Output Program



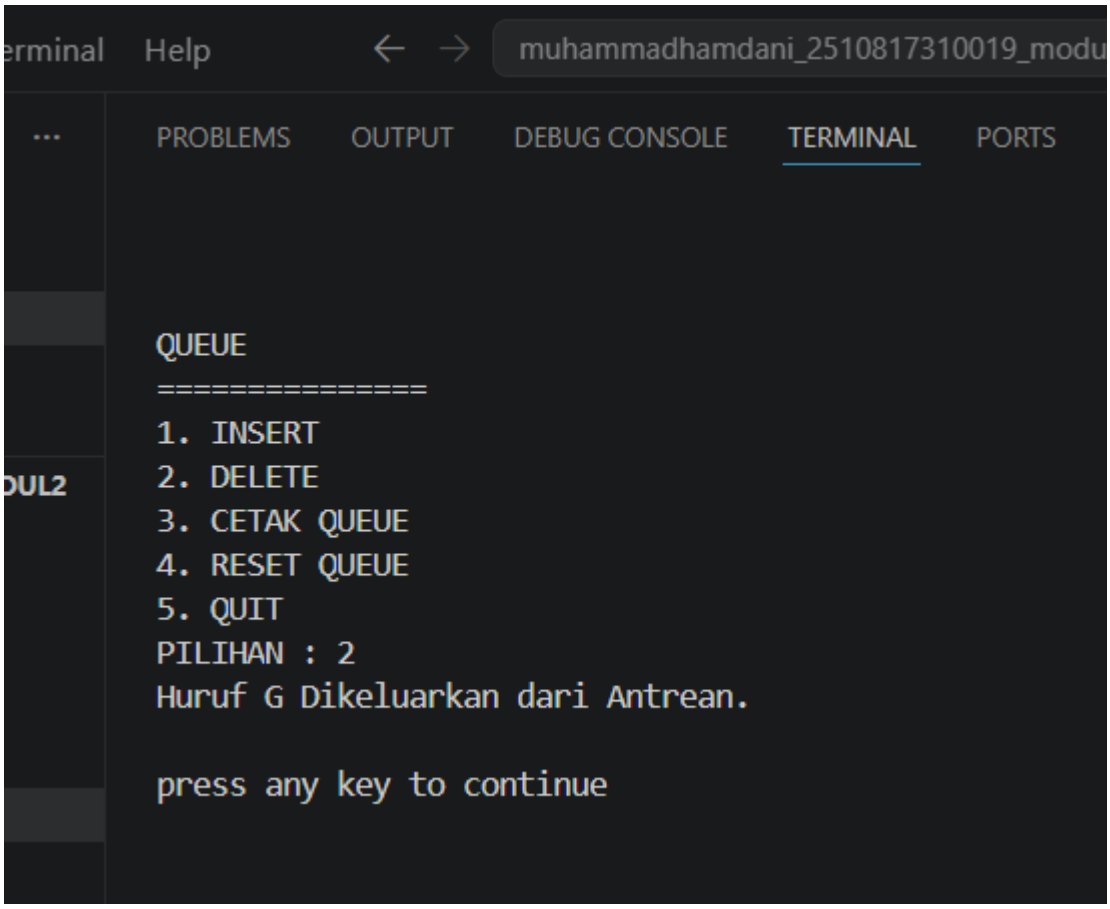
GAMBAR 6. Tampilan awal program saal dijalankan



GAMBAR 7. Memasukkan Huruf ke Dalam queue



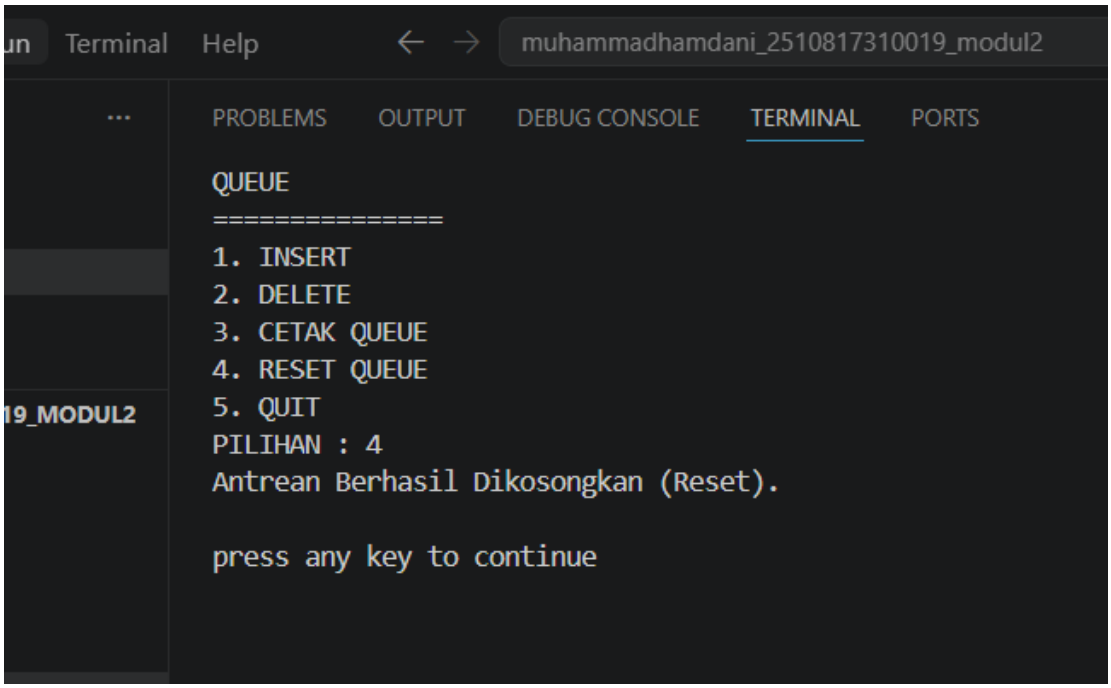
GAMBAR 8. Menampilkan Queue Yang Sudah Dimasukkan



GAMBAR 9. Melakukan Delete Pada Huruf yang Sudah Dimasukkan

```
Terminal   Help   ←  →   muhammadhamdani_2510817310019_m

...   PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

Huruf G Dikeluarkan dari Antrean.

press any key to continue

MODUL2

QUEUE
===============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. RESET QUEUE
5. QUIT
PILIHAN : 3
Queue ke- 1 = E
Queue ke- 2 = N
Queue ke- 3 = J
Queue ke- 4 = I

press any key to continue
```

GAMBAR 10. Tampilan Queue Setelah Melakukan Delete



GAMBAR 11. Tampilan mereset queue

GAMBAR 12. Tampilan Setelah melakukan reset dan Tampilan memilih quit

C.Pembahasan

Baris [1] sampai [3] terdapat perintah #include untuk memanggil *library*. <iostream> digunakan sebagai standar input-output. Kita menggunakan <stdlib.h> untuk fungsi atoi yang mengubah tipe data string dari input menu menjadi integer, serta untuk manajemen sistem.

Baris [5] terdapat #define n 10 yang berfungsi untuk membuat konstanta global. Variabel n didefinisikan dengan nilai 10, yang berarti kapasitas maksimal antrean adalah 10 elemen.

Baris [7] sampai [12] merupakan bagian deklarasi prototipe fungsi. Di sini terdapat fungsi INSERT(), DELETE(), CETAKLAYAR(), Inisialisasi(), dan RESET(). Deklarasi ini penting agar *compiler* mengenali fungsi-fungsi tersebut sebelum dipanggil di dalam fungsi main.

Baris [14] sampai [17] mendeklarasikan variabel global. int F adalah *Front* (depan) dan int R adalah *Rear* (belakang). char Q[n] adalah array tempat menyimpan data antrean.

Baris [19] sampai [51] adalah fungsi main() yang berisi alur logika program. Di sini digunakan do-while untuk menampilkan menu secara berulang. Input menu diambil menggunakan char PILIHAN[2] lalu dikonversi menggunakan atoi(PILIHAN) ke variabel PIL untuk diproses dalam blok switch-case. Program ini memberikan 4 opsi utama (Insert, Delete, Cetak, Reset) dan opsi ke-5 untuk keluar.

Baris [53] sampai [61] berisi fungsi Inisialisasi() dan RESET(). Keduanya memiliki logika yang sama yaitu mengatur nilai F dan R menjadi -1. Nilai -1 menandakan antrean benar-benar kosong dan pointer indeks berada di luar jangkauan array.

Baris [63] sampai [72] adalah fungsi INSERT(). Fungsi ini menerapkan logika Enqueue. Jika nilai R belum mencapai n-1, maka R akan bertambah (increment) dan data dimasukkan ke indeks tersebut. Jika antrean baru pertama kali diisi, maka F juga diatur ke posisi 0.

Baris [74] sampai [82] adalah fungsi DELETE(). Fungsi ini menerapkan logika Dequeue. Sesuai prinsip FIFO (First In First Out), data yang diambil adalah data yang

berada pada posisi depan (F). Setelah data diambil, nilai F akan bertambah (increment) untuk menunjuk ke elemen berikutnya.

Baris [84] sampai [94] adalah fungsi CETAKLAYAR(). Fungsi ini akan melakukan perulangan dari indeks F sampai R untuk menampilkan seluruh isi antrean. Jika posisi F sudah melampaui R, maka program secara otomatis mendeteksi bahwa antrean telah kosong.

TAUTAN GIT HUB

https://github.com/2510817310019-maker/Laprak-Algoritma-dan-Struktur-Data-muhammadhamdani-2510817310019